

# CodeFab Interpreter 프로젝트 기능 추가 요청

CONTENTS

# 목차

- |           |                       |
|-----------|-----------------------|
| CHAPTER 1 | 기능 추가 미션 소개           |
| CHAPTER 2 | [추가] function 관련 요구사항 |
| CHAPTER 3 | [추가] 정적 배열 구현         |
| CHAPTER 4 | [추가] 실행전 최적화          |
| CHAPTER 5 | [추가] 공장 제어 셀 제작       |
| CHAPTER 6 | 기능 추가 미션, 유의사항        |

Chapter 1

# 기능 추가 미션 소개

기능 추가, 리팩토링 미션 개요

## 3 ~ 4일차 미션

기능 확장을 염두에 두고 클린하게 리팩토링한 구조 위에,  
실제 기능을 추가해보자

### 미션

- function 관련 요구사항
- 정적 배열 구현
- 실행 전 최적화
- 공장 제어 셀 제작

Chapter 2

# [추가] function 관련 요구사항

추가 요구 사항 mission

# function 기본 구현

함수 선언과 호출 기능을 추가로 구현한다.

```
Func add(a, b) {  
    return a + b;  
}  
  
ret = add(3, 7);  
print ret;
```

## 지원 범위

| 기능             | 예시                                                                     |
|----------------|------------------------------------------------------------------------|
| 함수 선언          | Func add(a, b) { ... }                                                 |
| 함수 호출, 매개변수 전달 | add(1, 2);                                                             |
| return 처리      | return ; → null 값 반환<br>ret = add(1, 2); → ret 에 return 값 반환           |
| 재귀 호출          | Func fact(n) {<br>if(n <= 1) return 1;<br>return n * fact(n - 1);<br>} |

# function 관련 오류 검사

함수에 관련한 오류를 검사한다.

- 오류 메시지를 출력하고 오류 발생을 시킨다

## 지원 범위

| 기능                | 예시                                                        |
|-------------------|-----------------------------------------------------------|
| 함수 외부에서 return 사용 | return 5; → 함수 외부에 작성                                     |
| 파라미터 이름 중복        | Func foo(a, a) { ... } → 변수 a 이름 중복                       |
| 함수가 아닌 대상 호출      | var x = "hello";<br>x( ); // 문자열을 호출한 경우                  |
| 인자 개수 불일치         | Func foo(a, b, c) { ... } → 파라미터 3개<br>Foo(1, 2); → 인자 2개 |

Chapter 3

# [추가] 정적 배열 구현

추가 요구 사항 mission



# 정적 배열

고정 크기 배열을 지원한다.

- 배열 생성 시 크기가 확정
- 인덱스를 통한 읽기 / 쓰기 지원

예시 코드

```
var arr = Array(3);           // 크기 3의 배열 생성 [null, null, null]
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
print arr[0];                 // 10 출력

var i = 2;
arr[i - 1] = 7;
```

# 필요한 Token 과 Expr

## Token 추가

- LEFT\_BRACKET ( [ ) : 인덱스 시작
- RIGHT\_BRACKET ( ] ) : 인덱스 종료

## Expr 추가

- 배열 읽기 표현식 추가 : 예) `v = arr[i];` **인덱스 읽기**
- 배열 쓰기 표현식 추가 : 예) `arr[i] = 7;` **인덱스 쓰기**

# 런타임 오류

실행중 배열의 잘못된 사용의 경우 류를 발생시킨다.

- 범위를 벗어난 인덱스 접근
- 인덱스가 숫자가 아닌 경우
- 배열이 아닌 대상에 [] 사용
- 배열 생성시 크기로 숫자가 아닌 값 대입

```
var arr = Array(3);           // [null, null, null]

print arr[5];                 // 인덱스 범위 벗어남
print arr["hello"]           // 인덱스는 반드시 숫자여야함

var x = 10;
print x[0] ;                  // index 접근은 오직 배열만 지원한다

var brr = Array("hi");       // 배열의 사이즈는 반드시 number
```

Chapter 4

# [추가] 실행전 최적화

추가 요구 사항 mission



# 상수 연산 최적화 - Checker Unit 활용

## 리터럴 상수 연산에 대해 매번 계산할 필요 없는 수식 합치기

- 루프 내 상수 표현식에서 성능 향상
- 표현식의 결과가 런타임 이전에 100% 확정할 수 있을때만 최적화 진행
- Before : 100 만번의 루프 동안 매번 복잡한 연산작업을 수행 → 실행시 800 만번 연산
- After : 수식을 리터럴로 교체해서 연산 없이 바로 사용 → 실행시 100 만번 연산

```
var total = 0;
var start = clock();

for (var i = 0; i < 1000000; i = i + 1) {
    total = total + (1 - 2 * 3 * 4 * 5 / 6 + 7 + 8 + 9) % 1000 % 30;
}
```

→ 리터럴 5 로

# 최적화 여부 Test

Test Double 을 사용해서 2 가지 최적화를 검증해본다.

- 정적 바인딩
- 수식 합치기

## 최적화 검증 예시

정적바인딩 : 스코프를 거슬러 올라가지 않고 계산된 위치로 즉시 접근했는지 검증

수식 합치기 : 계산 횟수가 N 회에서 0회로 줄었는지 검증

Chapter 5

# [추가] 공장 제어 웹 제작

추가 요구 사항 mission



공장 제어 셸은 Interpreter Factory 를 운용하고 점검하는 인터페이스

- **모드** : Interpreter Factory 를 실행하는 방법을 선택
  - 프롬프트 모드(REPL), 파일 모드, 디버그 모드

## 프롬프트 모드(REPL Mode)

사용자가 소스를 한 줄씩 직접 입력하는 대화형 실행 모드  
기존에 구현한 모드로 파일모드와 구분해서 실행합니다.

### 동작 방식

- > 프롬프트가 표시되면 코드 한줄 입력 후 Enter
- 전역 변수 저장소는 세션 종료 전까지 유지
- exit 또는 quit 입력 시 종료

```
$ ./factory  
> var a = 3;  
> var b = 7;  
> print a + b;  
10  
> █
```

동작 예시

## 파일 모드

소스코드 파일을 읽어 실행하는 모드

```
$ ./factory run <파일경로>  
$ ./factory run ./scripts/hello.txt
```

## 구현 요구사항

- 파일이 존재하지 않을 경우 명확한 오류 메시지 출력
- 실행 중 런타임 오류 발생 시 오류 발생 줄 번호와 함께 출력
- 오류 발생시 오류 메시지 출력 후 즉시 종료

## 동작 예시

# 디버그 모드 - 1

## 디버그 모드

소스 코드를 한 Stmt 단위로 멈추며 실행 상태를 점검하는 모드

## 진입 방법

```
$ ./factory debug <파일경로>  
$ ./factory debug ./scripts/test.txt
```

## 구현 요구사항

- stepping 단위는 Stmt 기준
- watch 는 변수 저장소에서 직접 조회하여 출력

# 디버그 모드 – 2 Stepping

Stepping 은 파이프 라인을 Stmt 단위로 정지하며 단계별 실행을 한다.

| 명령어          | 설명                         |
|--------------|----------------------------|
| step         | 현재 Stmt 실행 후 다음 Stmt 에서 정지 |
| next         | 현재 Stmt 실행 (블록 내부로 진입 X)   |
| break <줄번호>  | 해당 줄에 breakpoint 설정        |
| Breakpoints  | 현재 설정된 breakpoint 목록 출력    |
| remove <줄번호> | breakpoint 해제              |
| continue     | 다음 breakpoint 까지 실행        |

예시

```
[DEBUG] 소스코드 로딩: ./scripts/test.txt
[DEBUG] 1번째 줄에서 정지 → var a = 3;
> step
[DEBUG] 2번째 줄에서 정지 → var b = a + 1;
> break 7
[DEBUG] 7번째 줄에 breakpoint 설정
> continue
[DEBUG] 7번째 줄에서 정지 (breakpoint) → print a;
```

# 디버그 모드 – 3 Watch Variable

## watch variable

실행 정지 시점마다 관찰중인(지정한) 변수의 현재 값을 자동으로 출력한다.

| 명령어           | 설명                                      |
|---------------|-----------------------------------------|
| watch <변수명>   | 해당 변수를 감시 목록에 추가                        |
| unwatch <변수명> | 감시 목록에서 제거                              |
| watches       | 현재 감시 중인 변수 목록과 값 출력 ( 가장 인접한 스코프의 변수 ) |
| inspect       | 현재 스코프의 모든 변수와 값 출력                     |

## 예시

```
> watch a
[WATCH] 'a' 감시 등록
> step
[DEBUG] 5번째 줄에서 정지 → a = a + 1;
[WATCH] a = 3
> step
[DEBUG] 6번째 줄에서 정지 → print a;
[WATCH] a = 4
```

```
> inspect
—— 현재 스코프 변수 —————
[로컬] b = 10 (Number)
[로컬] flag = true (Boolean)
[전역] a = 4 (Number)
```

Chapter 6

# 기능 추가 미션, 유의사항

미션 수행시, 유의사항

## 유의사항

1. 3일차 부터는 TDD로 개발하지 않아도 됩니다.
2. 기존 UnitTest는 유지보수를 되어야 합니다.
3. 추가 기능에 대한 UnitTest는 생성되어야 합니다.
4. Custom Language에 대한 사용 방법을 README.md에 명시합니다.
  - 기타 특이사항도 README.md에 명시합니다.



## 디자인 패턴을 사용시, 추가 점수

디자인 패턴은 여러 곳에서 발생할 수 있는 문제를 해결하는 일반화된 모범 사례입니다. 이를 클린코드의 Best Practice 라고 합니다. 따라서, GoF 디자인패턴을 사용하면 약간의 추가 점수가 주어집니다.

- 예시)

Visitor Pattern,  
Interpreter Pattern ,  
Command Pattern,  
Strategy Pattern 등